



۸۱۰۱۰۱۴۹۲



نوت ۲۳ آذر

شبکه‌های کامپیوتری

نام و نام خانوادگی: علیرضا کریمی

فصل پنجم: لایه *Network* بخش *Control – Plane* اسلاید ۶۴ تا ۸۹ [۱]

این خلاصه حاوی خلاصه ای از تدریس استاد در کلاس هم هست.

در روش قدیمی و سنتی، طراحی به صورت *monolithic* بود و روترها همه چیز داشتند. از انواع الگوریتم‌های روتینگ و فورواردینگ و سخت افزارها و غیره. درواقع یم سیستم عامل *IOS* روی روترها بود که همه این موارد را هندل میکرد. پس پیچیدگی طراحی در روتر زیاد بود.

بعد از آن، جعبه‌های میانی *middlebox*ها آمدند (اول به صورت نرم افزاری و بعد به صورت سخت افزاری) که میتوانستیم به کمک آنها کارهایی غیر از روتینگ، مثل *firewall* و *NAT* و *load balance* و غیره را هندل کنیم.

در روش‌های سنتی، *per router control plane* داشتیم و الگوریتم روتینگ به صورت *distributed* بود و روترها نیاز بود که با هم تعامل داشته باشند.

تصمیمات لایه *data path* بسیار سریع باید انجام شود (چون به یک *lookup* نیاز دارد) و تصمیمات در *control panel* کندتر است.

آقای کاسادو، مرز بین *data path* و *control path* را از هم جدا کرد. و مفهوم *SDN* را معرفی کرد.

$$SDN = \text{software defined network}$$

در این روش *data path* از *control path* جدا شده و توابع مربوط به *control path* از روترها خارج شده و پیاده سازی مربوطه به الگوریتم‌های روتینگ به جای روتر، در یک *remote controller* خارجی انجام میشود که حالا خود این میتواند یک *data center* باشد و یک سرور باشد و یا به صورت حتی خودش هم به صورت *distributed* طراحی شده باشد.

remote controller گراف شبکه را دارد و طبق آن هر چند وقت یک بار، عملیات روتینگ را انجام میدهد و با ساخت جداول مربوطه مثل جداول *forwarding* یا حتی *flow – based table* و دادن این اطلاعات به روترهای مربوطه از طریق $CA = \text{control agent}$ ها در هر روتر، این اطلاعات را به آنها میدهد.

یکی از کارهای مهم و جانبی دیگر آن، *network management* و کمک به این کار است. و برای کنترل ترافیک و *flow* کاربرد دارد. درواقع *SDN*، *misconfiguration* را کم میکند.

قبلا همه روترها *programmable* بودند و همه کارها رو انجام میدادند. بعد اومدن یک سری سویچ *programmable* طراحی کردن که بعضی از محاسبات رو خودش انجام میداد و لازم نبود اون کارها رو به *control plane* بفرسته و یا لودش روی اینترنت بیاد. اینطوری به ترافیک هم کمک میکرد و اینگونه *programmable data plane* درست شد.

قبلا یک *mainframe* بود که همه چیز از جمله سخت افزار و سیستم عامل و اپلیکیشن و غیره را داشت. و اینها به صورت *vertically* روی هم قرار میگرفتند الان این سخت افزارها و سیستم عامل و اپلیکیشن ها از هم جدا شدند و هر کدام *programmable* هستند. و میتوان آنها را به صورت *horizontally*، اسکیل کرد. محدودیت های روش های سنتی:

در روش های سنتی، نمیتوان وزن لینک ها را تغییر داد تا مسیر دیگری جایگزین شود. شاید بخواهیم یک قسمتی از *packet* ها از یک مسیر و بقیه از مسیر دیگری برود. این امکان در روش سنتی فراهم نیست.

در روش سنتی، نمیتوان بین ترافیک ها از *flow* های مختلف، تمییز قائل شد و متفاوت باهاشون رفتار کرد و مسیر متفاوتی براشون در نظر گرفت.

در *SDN*، از *flow table* به جای *forwarding table* استفاده میکنیم و میتوانیم و چالش ها را حل کنیم. میتونیم بعضی از *functionality* های رو که لازم داریم، به صورت نرم افزاری روی *SDN switch* ها *embedd* کنیم. در *openflow* یک *API* داریم که کنترلر از طریق اون با سویچ ها صحبت میکنه(?) در واقع *SDN controller* از طریق *north bound API* (که معمولا هم *web - based* هستند) با اپلیکیشن های مختلف مثل *routing* و *access network* و *load balance* و غیره در ارتباط است و از طریق *south bound API* با سویچ ها در *data plane* در ارتباط است.

SDN controller یجور *OS* شبکه روی سویچ هاست و اون ها رو مدیریت میکنه و *state* شبکه رو ذخیره میکنه. در *SDN controller* اطلاعات مختلفی مثل *host info* و *switch info* و *link - state info* که از سویچ ها جمع آوری شده و ذخیره شده است در یک دیتابیس توزیع شده. روی اون اطلاعات میتوان جداول فوردواردینگ را ساخت و اطلاعات آماری را گردآوری کرد.

در بخش بالایی *API SDN controller* های مختلف برای ارتباط این کنترلر با کاربرد های کنترل در سطح بالاتر است. مثلاً شبکه را به صورت یک گراف در اختیار لایه بالاتر قرار میدهد. یا برآیند کاربرد های مختلف مثل *web-based* از *restful API* استفاده میکند. یک *API* هم *intent-based* است که به صورت *agentic* است و اوپراتور، مقصود خود را به صورت سطح بالا بیان میکند و اون *agent* آن را کامپایل میکند و به سطح پایینتر میدهد. خود *SDN controller* هم از ۳ بخش تشکیل شده است. به گفته استاد در کلاس، *mininet* یک *emulator* هست برای پیاده سازی *SDN* اپلیکیشن ها به صورت *unbundled* است یعنی هر کی میتونه *develop* کنه

openflow protocol

قبلاً امن نبود.

۳ نوع پیغام دارد:

۱. پیغام های *controller - to - switch*
 ۲. پیغام های آسنکرون یا پیام های سویچ به کنترلر
 ۳. پیغام های سنکرون مثل *timestamp* ها
- هم یک پورتکل ارتباطی است و هم یک *API* برای مشخص کردن اعمالی که در جدول *flow* انجام میشود.

پیغام های *controller - to - switch*

۱. *feature*: کنترلر یک سری اطلاعات رو از سویچ ها، کوئری میکند.
۲. *configures*: پارامترهای پیکر بندی را سویچ را میپرسد و تنظیمی میکند.
۳. *modify - state*: مدخل های جدول *flow* را اضافه یا کم یا اصلاح میکند
۴. *packet - out*: کنترلر میتواند بسته مورد نظر را از یک پورت سویچ به بیرون بفرستد.

پیغام های *switch - to - controller*

۱. *packet - in*: سویچ اطلاعات کنترلی را به کنترلر میفرستد. در مقابل *packet - out*
 ۲. *flow - removed*: بعضی از مدخل های سویچ را پاک میکند
 ۳. *port - status*: کنترلر رادرمورد تغییر در یک پورت سویچ آگاه میکند.
- پس اوپراتور های شبکه مستقیم این سویچ ها را برنامه نویسی نمیکنند و با یک *abstraction* بالاتر و از طریق کنترلر این کار را میکنند.

در اسلاید ۸۱، یک مثال از ارتباط بین لایه های مختلف بین *data and control plane* آمده است.

سوییچ‌ها اطلاعات *port – status* از طریق *openflow* به کنترلر می‌دهد. و اون هم *link – state info* را آپدیت میکند و بعد یک *graph API* به کاربرد الگوریتم روتینگ دایکسترا در لول بالاتر می‌دهد. بعد از انجام روتینگ، اطلاعات لازم در *link – state info* آپدیت شده و *flow table* هم آپدیت میشود و این جدوال جدید *flow* را در سوییچ‌هایی که نیاز به آپدیت دارن میدن.

در اسلاید درس از یک کنترلر دیگر به اسم *OpenDayLight(ODL)* رو هم آورده ازش ولی توضیحی توی وویس نداده.

ONOS

کنترلر معروف دیگر *OSON* است و کاربرد های کنترلی از کنترلر جدا هستند و از طریق *northbound API* با آن ارتباط برقرار میکند

از طریق *rest API* ارتباط برقرار میکند و همچنین از *intent API* که برای کاربرد های هوش مصنوعی و *LLM* هاست استفاده میکند.

یعنی مشخصات پیکربندی را در سطح خیلی بالا و در سطح *what* بیان میکند و نه *how* و خودش لازم نیست درگیر جزئیات باشد.

هسته کنترلر به صورت توزیعی است. هسته کنترلر که در بخش وسطی شکل است، شامل قواعد توپولوژی و مسیر ها و لینک ها و *flow* ها و دیوایس و آمار ها وجود دارد. این توزیعی بودن هسته، باعث پایایی سرویس و کارایی مقیاس پذیر میشود.

یک سری هم *southbound API* هست که در شکل اسلاید آمده.

چالش های SDN

باید *control plane* را *hardening* کرد.

باید به صورت توزیع شده، به صورت *reliable* و *dependable* و *performance – scalable* باشد و *secure* باشد و نسبت به *robustness*، *failure* داشته باشد.

بهتر است که *security* و *depenedability* را از همان روز اول در نظر بگیریم.

در شبکه های 5G و 6G، *SDN* خیلی کاربرد دارد.

بعضی از نیاز هایی که از *SDN* می‌خواهیم و پیاده سازی سختی دارند، از انواع نیازمندی های *mission – specific*

مثل *real – time* و *ultra – reliable* و *ultra – secure*

میتوان *SDN* را در سطح بیشتر از یک *AS* هم اسکیل کرد.

آینده SDN

گفتیم در SDN به صورت منطقی مرکزی محاسبه می‌شود در مقابل روتر ها که توزیع شده محاسبه می‌شود. عمل دیگری غیر از محاسبه فورواردینگ که میتوان توسط SDN پیاده سازی کرد، *congestion control* میتواند باشد.

کنترلر طبق سطح ازدحام گزارش شده، نرخ های ارسال را به روتر ها گزارش می‌کنند. حالا سوال مهم این است که این کاربرد ها چگونه به حالت SDN گسترش پیدا کنند.

ICMP = internet control message protocol

برای انتقال اطلاعات سطح شبکه بین هاست ها و روتر ها برای گزارش خطاهایی مثل اینکه فلان هاست های از دسترس خارج شده و یا شبکه و پورت و پروتکل ناموجود. پیام های *echo* و درخواست و پاسخ که در *ping* استفاده می‌شود. پیام های *ICMP* خودش مربوط به لایه شبکه است و بالای *IP* است و از طریق دیتاگرام های لایه *IP* حمل می‌شود. پس یجورایی توش لوپ وجود داره و اگر در خود پیام *IP* خطا رخ بده، نمیره و متوجه نمیشم. (چون پیغام خطا لایه شبکه داره خودش با دیتاگرام لایه شبکه منتقل میشه و همون دیتاگرام میتونه خطا بخوره و یجورایی این موضوع میتونه لوپ بخوره)

پیام های *ICMP* شامل تایپ و کد (دو تا عدد هستن که توی جدول اسلاید ۸۸ اومدن) و به علاوه ۸ بایت اول اون دیتاگرامی که دارای خطا شده است.

مثلا تایپ ۳ مربوط به *unreachable* ها است. پورت، شبکه و هاست و غیره

کاربرد پروتکل *ICMP* در برنامه *traceroute*: مبدا مجموعه ۳ تایی از پیام های *UDP* را می‌فرستد، و هر مجموعه به ترتیب $TTL = 1, 2, 3, \dots$ است. گفتیم که به هر روتر برسد، یک واحد از *TTL* کم می‌شود و به بعدی می‌فرستد. وقتی که به روتر *n* رسید که *TTL* صفر شد و تموم شد، اونجا روتر پیغام را *discard* و حذف میکند و یک خطای *ICMP* با تایپ ۱۱ و کد صفر، می‌فرستد. در این پیام ممکن است نام روتر و آدرس *ip* روتر ذکر می‌شود و احتمالا به این دلیل است که بعضی از روتر های سر راه *ICMP* را بسته باشند. مبدا از روی اون، *RTT* تا روتر *n* را می‌فهمد. شرط خاتمه عملیات *traceroute*:

مبدا یک پورت ناموجود و یا کمتر محتمل را انتخاب میکند و وقتی که این پیام های *udp* به مقصد رسیدند، مقصد پیام خطای *ICMP* از تایپ ۳ و کد ۳ میدهد که یعنی *port unreachable*. اینگونه با دریافت این پیام، مبدا *RTT* تا مقصد را می‌فهمد و متوقف می‌شود.

پروتکل *SNMP* هم برای مدیریت شبکه است که مربوط به اسلاید های دور بعدی می‌شود

مراجع

- [1] “ch5-slide-64-to-89,” <https://drive.google.com/drive/folders/>, 2022.